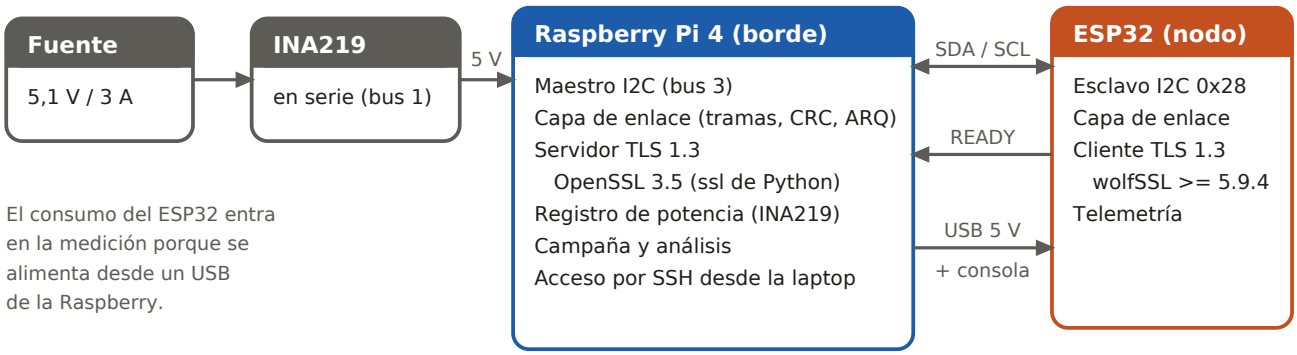
Guía 00 · Mapa del trabajo práctico

Qué se construye, en qué orden, qué comprar y qué ya está verificado.

Objetivo	Tener el mapa completo del trabajo práctico: qué se construye, en qué orden y cómo se sabe que cada semana terminó.
Dedicación estimada	Lectura de 20 minutos; vuelvan a esta guía cuando se pierdan.
Antes de empezar	Propuesta aprobada (v4) y este paquete: **12 guías** + **codigo_integradora.zip**.
Al terminar tendrán	Claridad sobre el plan, las fechas sugeridas y las compras que deben hacer ya.

1. Qué van a construir

Un banco de pruebas donde un **nodo sensor (ESP32)** abre sesiones TLS 1.3 con una **unidad de borde (Raspberry Pi 4)** usando el **bus I2C como único enlace**. En cada sesión se compara el intercambio de claves **híbrido X25519MLKEM768** con el **clásico X25519**, y se miden tiempo, tráfico en el bus, memoria y energía del sistema (sensor INA219). También se demuestra con un analizador lógico que la telemetría viaja cifrada y que un atacante que altera tramas es detectado.



Semana 8: analizador lógico conectado a SDA/SCL y GND

2. Cómo usar estas guías

- Cada guía tiene: **ficha** (objetivo y requisitos), **pasos** con comandos, recuadros **«Si falla»**, y al final el **criterio de cierre** (lista para marcar) y **qué subir al repositorio**.
- **No avancen a la semana siguiente sin cumplir el criterio de cierre**. Si algo no se puede cumplir, anótenlo en `docs/decisiones.md` con la razón: eso también es un resultado para la tesis.
- Todo el código ya está escrito y probado en un simulador (ver sección 5). Su trabajo es montarlo, ejecutarlo en el hardware real, verificar y medir. Si algo del hardware obliga a cambiar código, pídanme ayuda con el mensaje de error exacto.
- Lleven una **bitácora** en `docs/bitacora.md`: fecha, qué hicieron, qué falló, cómo lo resolvieron. Al final de cada sesión: `git add, commit y push`.

3. Cronograma práctico (12 semanas)

Las fases son las de la propuesta aprobada. Las fechas son sugeridas desde esta semana; ajústenlas a su calendario.

Sem	Fechas sugeridas	Fase (propuesta)	Objetivo práctico	Evidencia que queda
1	12–18 oct	Revisión y validación	Base del laboratorio, seguridad, OpenSSL 3.5	entorno_*.txt, repo reorganizado
2	19–25 oct	Revisión y validación	ESP-IDF, wolfSSL 5.9.4, simulador en la Pi	benchmark ESP32, sim_*.txt
3	26 oct–1 nov	Enlace I2C y sensor	Cableado, bus dedicado, eco a 100 kbit/s	eco_*.json
4	2–8 nov	Enlace I2C y sensor	Modo rápido, tamaño de trama, calibración INA219	calibración, reposo_*.json
5	9–15 nov	Capa de enlace y TLS clásico	Enlace confiable sin TLS	prueba_enlace_*
6	16–22 nov	Capa de enlace y TLS clásico	TLS 1.3 clásico sobre I2C	corridas clásicas
7	23–29 nov	Híbrido y confidencialidad	TLS 1.3 híbrido sobre I2C	corridas híbridas, tag v0.7
8	30 nov–6 dic	Híbrido y confidencialidad	Analizador lógico y ataques	capturas .sr, ataques_*.json
9	7–13 dic	Campaña de mediciones	Entorno congelado, piloto, tamaño de muestra	piloto_*, tamano_muestra.csv
10	14–20 dic	Campaña de mediciones	Campaña completa (2 bloques)	campana_* (.gz)
11	21–27 dic	Análisis	Estadística y figuras	resumen.csv, figuras
12	28 dic–3 ene	Informe final	Cierre técnico y paquete reproducible	release v1.0, dataset

4. Materiales y software

Compren esta semana (el envío tarda)

- **Analizador lógico USB de 8 canales, 24 MHz** (clon compatible con sigrok/PulseView, ~10 USD). Se usa en la semana 8.
- **2 resistencias de 4,7 kΩ** (pull-up del bus I2C del enlace), cables dupont macho-hembra, una protoboard pequeña.
- Si no lo tienen: **cable USB de datos** para el ESP32 (no solo de carga) y un **disipador o ventilador** para la Raspberry.

Ya tienen

- Raspberry Pi 4 con Raspberry Pi OS Lite 64 bits, acceso SSH, entorno virtual de Python.
- Sensor INA219 conectado y probado con multímetro. Repositorio en GitHub.
- ESP32: **confirmen el modelo exacto** (está impreso en el módulo metálico). Las guías asumen el **ESP32 DevKit clásico (ESP32-WROOM-32)**; con otro modelo solo cambian los pines (semana 3).

Versiones mínimas

Componente	Versión	Dónde
Raspberry Pi OS	Debian 13 (trixie), 64 bits	Raspberry
OpenSSL	3.5 o superior (trae ML-KEM y X25519MLKEM768)	Raspberry
Python	el del sistema (3.13 en trixie)	Raspberry
ESP-IDF	v5.5	laptop
wolfSSL	**v5.9.4-stable o superior** (ver sección 5)	laptop (firmware) y Raspberry (simulador)
PulseView (sigrok)	versión estable actual	laptop

5. Lo que ya está verificado (antes de tocar el hardware)

Todo el código del paquete se probó en un simulador que ejecuta **exactamente el mismo código C del ESP32** (capa de enlace y cliente TLS) contra el servidor de la Raspberry, con OpenSSL 3.5. Resultados:

- Handshake TLS 1.3 **clásico** e **híbrido** completos sobre el enlace con tramas, CRC y retransmisión: correctos. ClientHello de ~200 B (clásico) frente a ~1.380 B (híbrido).
- Las **5 pruebas de seguridad** pasan: sin TLS el texto de la telemetría se ve en el bus; con TLS no; y alterar un bit (con CRC recalculado) o reinyectar una trama hace que TLS rechace la sesión en ambos sentidos.
- La campaña, el registro de potencia, el cálculo de energía, la estadística y las figuras funcionan de punta a punta.

> **Hallazgo importante: usen wolfSSL 5.9.4 o superior**
>
> Con wolfSSL **5.8.2**, alrededor del **3 %** de los handshakes híbridos fueron rechazados por OpenSSL 3.5 (error BAD_ECPOINT): 10 de 300 en una prueba directa por TCP, sin el enlace de por medio. Con wolfSSL **5.9.4** (25 sep 2026) hubo **0 fallos en 1.000** handshakes. Las guías usan 5.9.4. Este dato sirve para la tesis: muestra un problema real de interoperabilidad en implementaciones post-cuánticas.

6. Estructura del repositorio (desde la semana 1)

Carpeta	Para qué
pi/	Programas de la Raspberry: enlace.py, servidor_tls.py, prueba_eco.py, potencia.py, medir_reposo.py, ataque.py, campana.py, tamano_muestra.py, analisis.py, sistema.py, conf/openssl_grupos.cnf
esp32/firmware/	Proyecto ESP-IDF del nodo: main.c (I2C esclavo, READY), enlace.c, cliente_tls.c, components/wolfssl (configuración de wolfSSL)
sim/	Esclavo simulado (mismo código C) para probar sin hardware
herramientas/	verificar_entorno.sh, cert_a_header.py, buscar_marcador_analizador.py
certs/	Certificado del servidor (servidor.pem se sube; servidor.key NUNCA)
data/raw, data/processed	Datos crudos (campaña) y procesados (análisis)
docs/	bitácora, decisiones, diccionario de datos, evidencias, figuras
legacy/	Scripts de la propuesta anterior (Kyber/Dilithium/RSA sueltos): se conservan, no se usan

7. Riesgos conocidos y plan B

Riesgo	Señal	Plan B
Alargamiento de reloj (clock stretching) de I2C en la Raspberry	tramas distintas en la prueba de eco	bajar velocidad, tramas de 32 B, o bus I2C por software (semana 3)
Memoria del ESP32 con ML-KEM	reinicio con «stack overflow»	subir la pila de la tarea TLS (semana 7)
OpenSSL < 3.5 en la Raspberry	verificar_entorno.sh no muestra ML-KEM	actualizar el sistema (semana 1); avisarme
Tiempo	una semana se alarga	recortar: varios nodos y extras quedan como trabajo futuro

Antes de empezar la semana 1

- ☐ Leyeron esta guía completa y tienen claras las 12 semanas.
- ☐ Pidieron el analizador lógico y las resistencias.
- ☐ Confirmaron el modelo exacto del ESP32 y lo anotaron.
- ☐ Ajustaron las fechas del cronograma a su calendario.